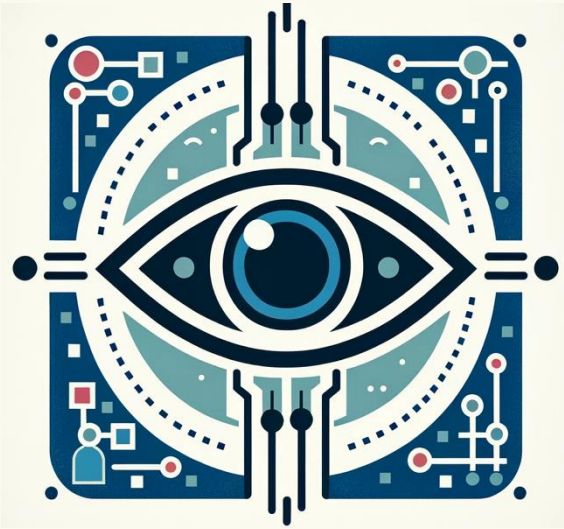


# COMPUTER VISION

## Warping

Luis F. Teixeira | M.EIC+M.IA | FEUP

# Recall: what types of image transformations can we do?

$F$

Filtering

$G(\boldsymbol{x}) = h\{F(\boldsymbol{x})\}$

$G$

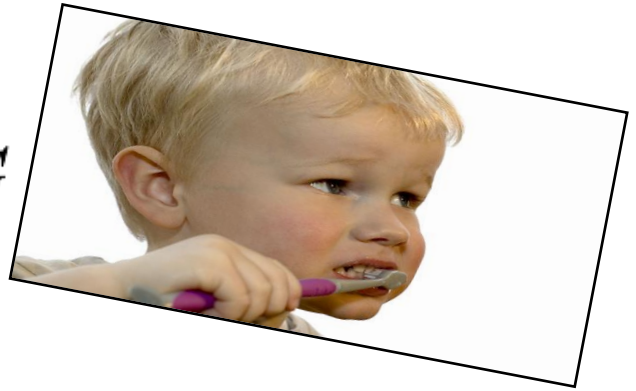
changes pixel *values*

$F$

Warping

$G(\boldsymbol{x}) = F(h\{\boldsymbol{x}\})$

$G$



changes pixel *locations*

# Parametric (global) warping

translation

rotation

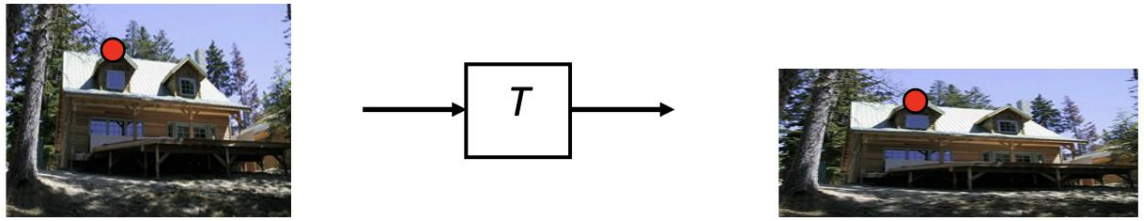
aspect

affine

perspective

cylindrical

# Parametric (global) warping




**p** = (x,y) **p'** = (x',y')

- Transformation T is a coordinate-changing machine:

$$p' = T(p)$$

- What does it mean that *T* is global?
  - Is the same for any point p
  - can be described by just a few numbers (parameters)
- Let's consider *linear* forms (can be represented by a 2D matrix):

$$\mathbf{p}' = \mathbf{T}\mathbf{p} \qquad \begin{bmatrix} x' \\ y' \end{bmatrix} = \mathbf{T} \begin{bmatrix} x \\ y \end{bmatrix}$$

# 2D Linear transformations

- Linear transformations are combinations of …
  - Scale,
  - Rotation,
  - Shear, and
  - Mirror

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

- Properties of linear transformations:
  - Origin maps to origin
  - Lines map to lines
  - Parallel lines remain parallel
  - Ratios are preserved
  - Closed under composition

# Defining the transformations

- Scaling $\mathbf{S} = \begin{bmatrix} s & 0 \\ 0 & s \end{bmatrix}$
- Rotation $\mathbf{R} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$
- Translation $\begin{aligned} x' &= x + t_x \\ y' &= y + t_y \end{aligned}$ $\mathbf{T}$ = ?

# Homogeneous coordinates

- (N+1)-dimensional notation for points in N-dimensional Euclidean space
- Allows us to express translation and projection as linear operations

**Normal coordinates**

$$v = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

**Homogeneous coordinates**

$$v = \begin{bmatrix} wx \\ wy \\ wz \\ w \end{bmatrix}$$

w is an arbitrary constant

$wx = w \cdot x$

**Example:**

$$v = \begin{bmatrix} 4 \\ -2 \\ 5 \end{bmatrix}$$

**w = 1** → $v = \begin{bmatrix} 4 \\ -2 \\ 5 \\ 1 \end{bmatrix}$

**w = -2** → $v = \begin{bmatrix} -8 \\ 4 \\ -10 \\ -2 \end{bmatrix}$

- To recover normal coordinates, divide the first N components by $(N+1)^{th}$

# Translation

- Using homogeneous coordinates:

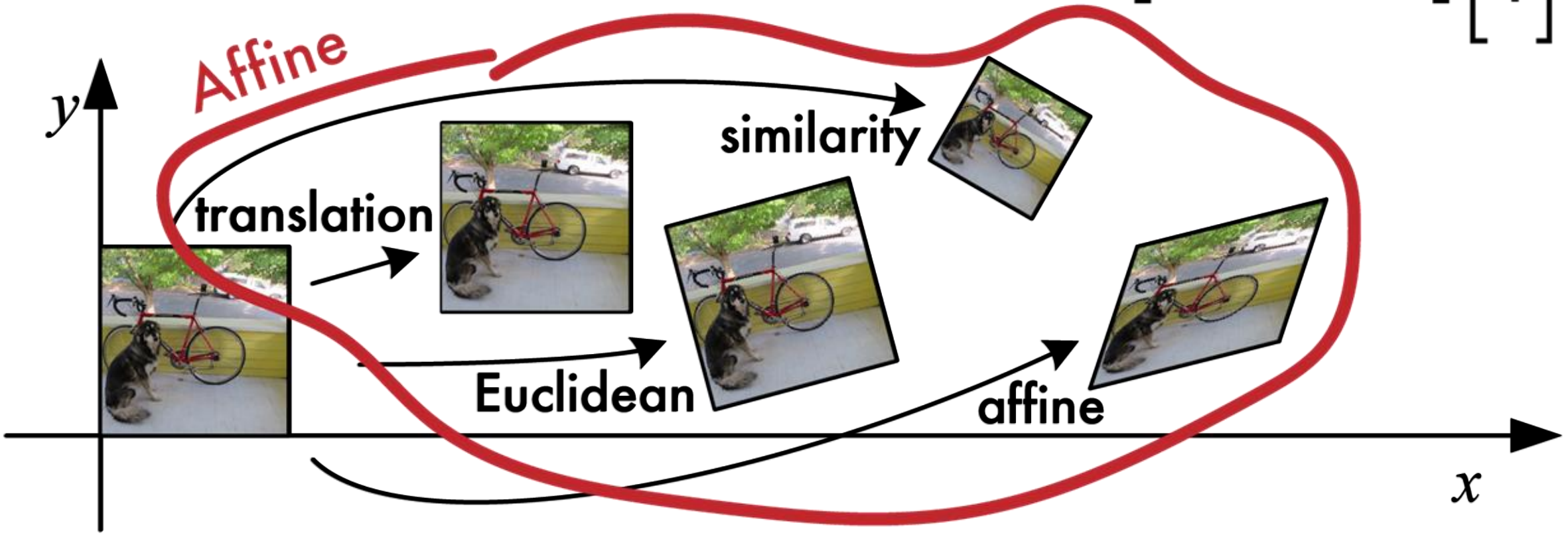
$$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \end{bmatrix}$$

$$\mathbf{T} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix}$$

# Affine: scale, rotate, translate, shear

General case of 2x3 matrix

$$\mathbf{x}' = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



# Affine transformations

- Say you want to translate, then rotate, then translate back, then scale.

  $$\mathbf{x'} = \mathbf{S}\,\mathbf{t}\,\mathbf{R}\,\mathbf{t}\,\bar{\mathbf{x}} = \mathbf{T}\,\bar{\mathbf{x}},$$

- If $\mathbf{T} = (\mathbf{S}\,\mathbf{t}\,\mathbf{R}\,\mathbf{t})$
- **T** is still affine transformation (i.e. combinations are still affine)
- But these are all 2x3, how to we multiply them together?

# Affine transformations

$$\bar{\mathbf{x}}' = \begin{bmatrix} 1 & 0 & dx \\ 0 & 1 & dy \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\bar{\mathbf{x}}' = \begin{bmatrix} \cos\theta & -\sin\theta & dx \\ \sin\theta & \cos\theta & dy \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\bar{\mathbf{x}}' = \begin{bmatrix} a & -b & dx \\ b & a & dy \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$\bar{\mathbf{x}}' = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

adapted from Joseph Redmon

# Affine transformations

- Affine transformations are combinations of …
  - Linear transformations, and
  - Translations

$$\begin{bmatrix} x' \\ y' \\ w \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

- Properties of affine transformations:
  - Origin does not necessarily map to origin
  - Lines map to lines
  - Parallel lines remain parallel
  - Ratios are preserved
  - Closed under composition

# Affine transformations

$$\mathbf{T} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

any transformation with last row [ 0 0 1 ] we call an *affine* transformation

$$\begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix}$$

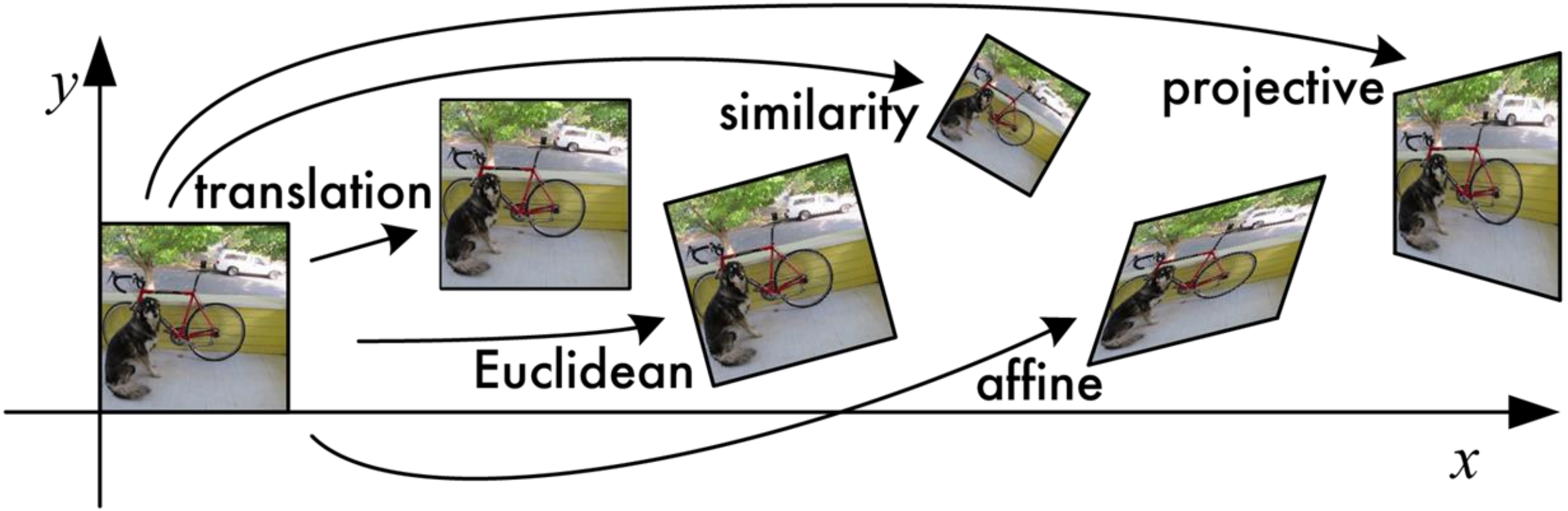
$$\begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix}$$

what happens when we mess with this row?

affine transformation

# Projective transform

- AKA perspective transform or **homography**



# Homography

- Homography is a general 3x3 matrix

$$\tilde{\mathbf{x}}' = \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{bmatrix} \begin{bmatrix} \tilde{x} \\ \tilde{y} \\ \tilde{w} \end{bmatrix} \qquad \tilde{\mathbf{x}}' = \tilde{\mathbf{H}}\,\tilde{\mathbf{x}}$$

# Homography

- Using homography to project point
  - Multiply $\tilde{\mathbf{x}}$ by $\tilde{\mathbf{H}}$ to get $\tilde{\mathbf{x}}'$

$$\tilde{\mathbf{x}}' = \tilde{\mathbf{H}}\,\tilde{\mathbf{x}}$$

  - Convert to $\bar{\mathbf{x}}'$ by dividing by $\tilde{w}'$

$$\bar{\mathbf{x}}' = \tilde{\mathbf{x}}' / \tilde{w}'$$

- Multiplication by scalar: equivalent projection
  - $3*\mathbf{H} \sim \mathbf{H}$

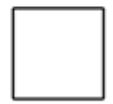
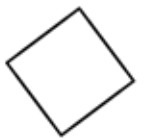
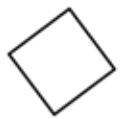
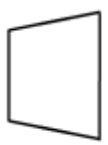
# Homography

- Homographies …
  - Affine transformations, and
  - Projective warps

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ w \end{bmatrix}$$

- Properties of projective transformations:
  - Origin does not necessarily map to origin
  - Lines map to lines
  - Parallel lines do not necessarily remain parallel
  - Ratios are not preserved
  - Closed under composition

# Transformations overview

| Transformation | Matrix | # DoF | Preserves | Icon |
|---|---|---|---|---|
| translation | $\begin{bmatrix} \mathbf{I} \mid \mathbf{t} \end{bmatrix}_{2\times3}$ | 2 | orientation |  |
| rigid (Euclidean) | $\begin{bmatrix} \mathbf{R} \mid \mathbf{t} \end{bmatrix}_{2\times3}$ | 3 | lengths |  |
| similarity | $\begin{bmatrix} s\mathbf{R} \mid \mathbf{t} \end{bmatrix}_{2\times3}$ | 4 | angles |  |
| affine | $\begin{bmatrix} \mathbf{A} \end{bmatrix}_{2\times3}$ | 6 | parallelism |  |
| projective | $\begin{bmatrix} \tilde{\mathbf{H}} \end{bmatrix}_{3\times3}$ | 8 | straight lines |  |

How can we **estimate the transformations parameters** when we don’t know them?

# Estimating affine transformation

A?

# Estimating affine transformation

- Have our matched points (as seen in the last lecture)
- Want to estimate **A** that maps from **x** to **x'**

  **xA** = **x'**

- How many degrees of freedom?
  - 6
- How many knowns do we get with one match? **mA** = **n**
  - 2

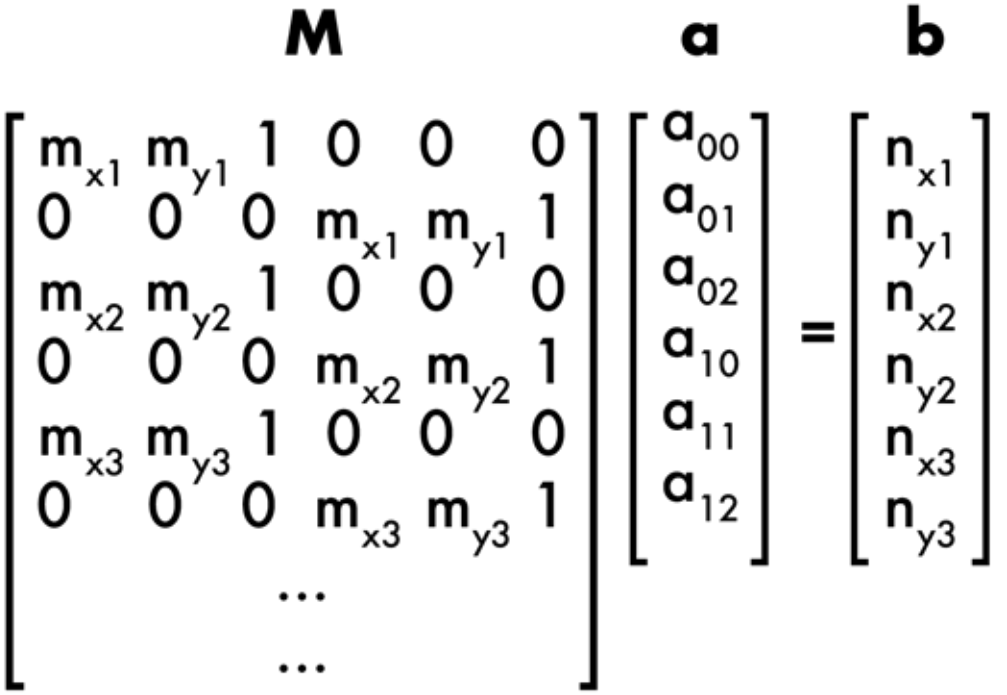
$$\mathbf{x}' = \begin{bmatrix} a_{00} & a_{01} & a_{02} \\ a_{10} & a_{11} & a_{12} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$n_x = a_{00} * m_x + a_{01} * m_y + a_{02} * 1$$
$$n_y = a_{10} * m_x + a_{11} * m_y + a_{12} * 1$$

# Estimating affine transformation

- Solve **M a** = **b**
  - $M^T M\, a = M^T b$
  - $a = (M^T M)^{-1} M^T b$

- Still works if overdetermined




This is the solution for linear least squares (i.e. minimise the squared error)

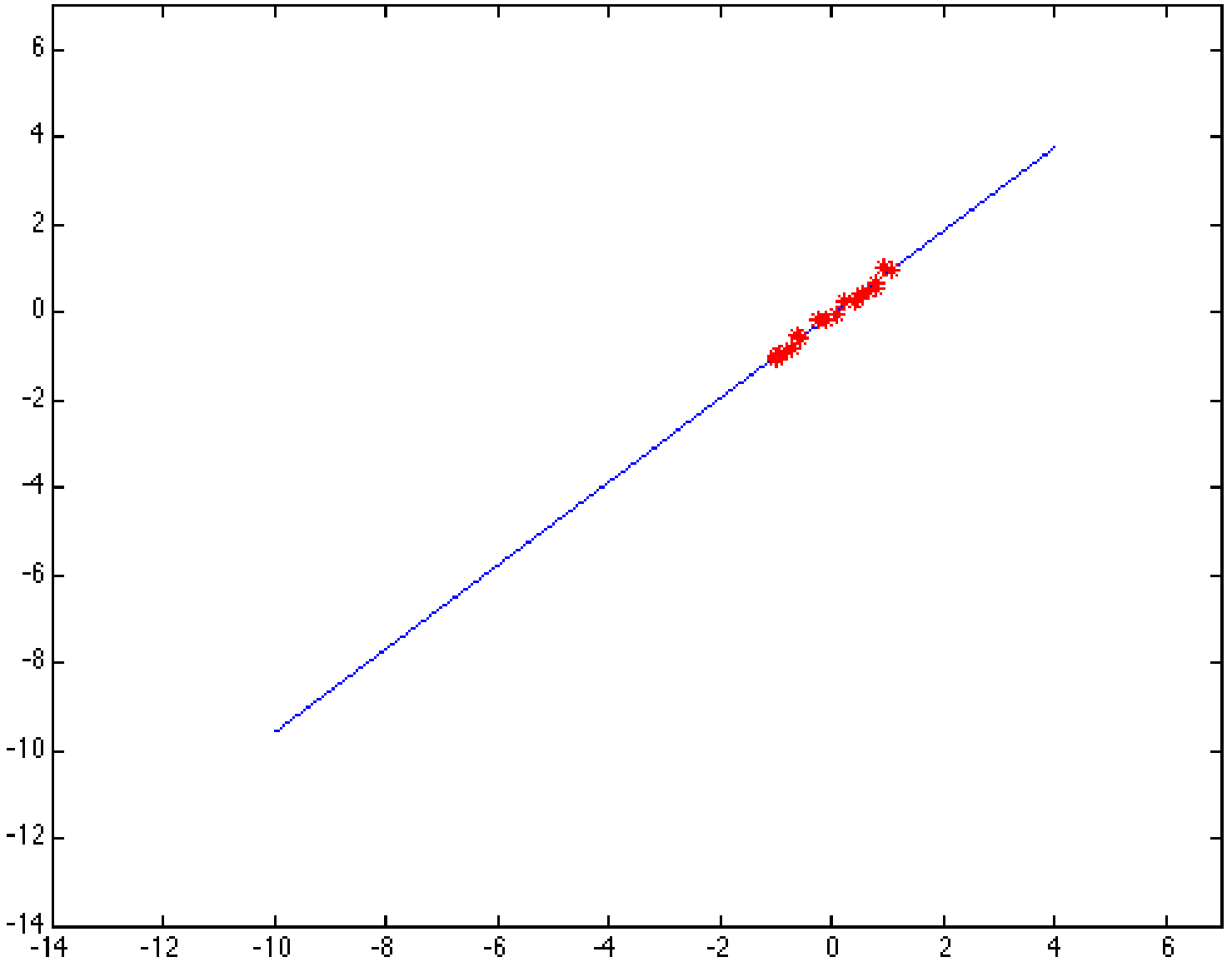
# Homography computation

- Similar to the estimation of the affine transformation
  - In this case, we need at least 4 matches

- Possible algorithm:
  - Given images A and B
    1. Compute **image features** for A and B
    2. **Match features** between A and B
    3. **Compute homography** between A and B using least squares on set of matches

  - What could go wrong?

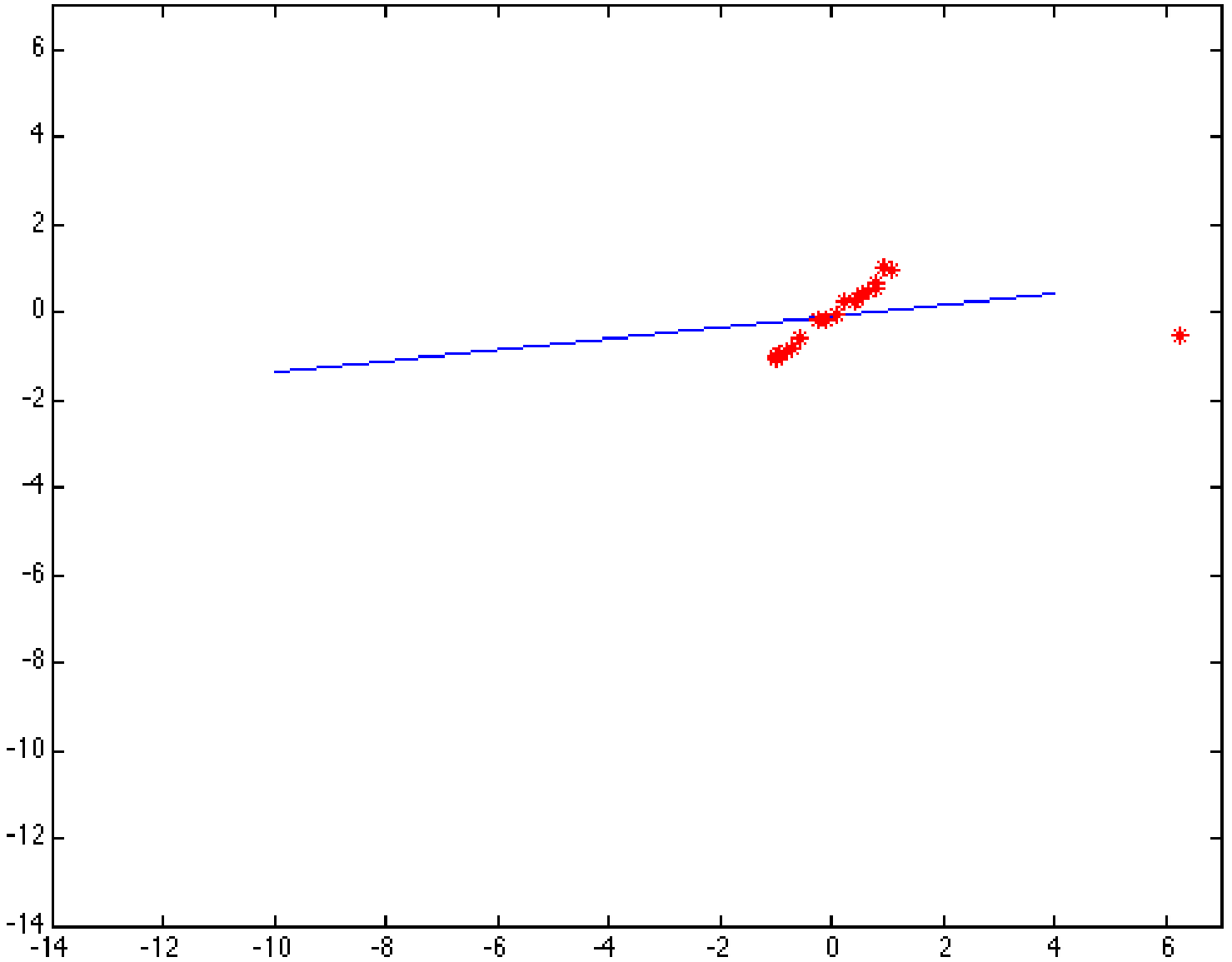
# How does least squares do?



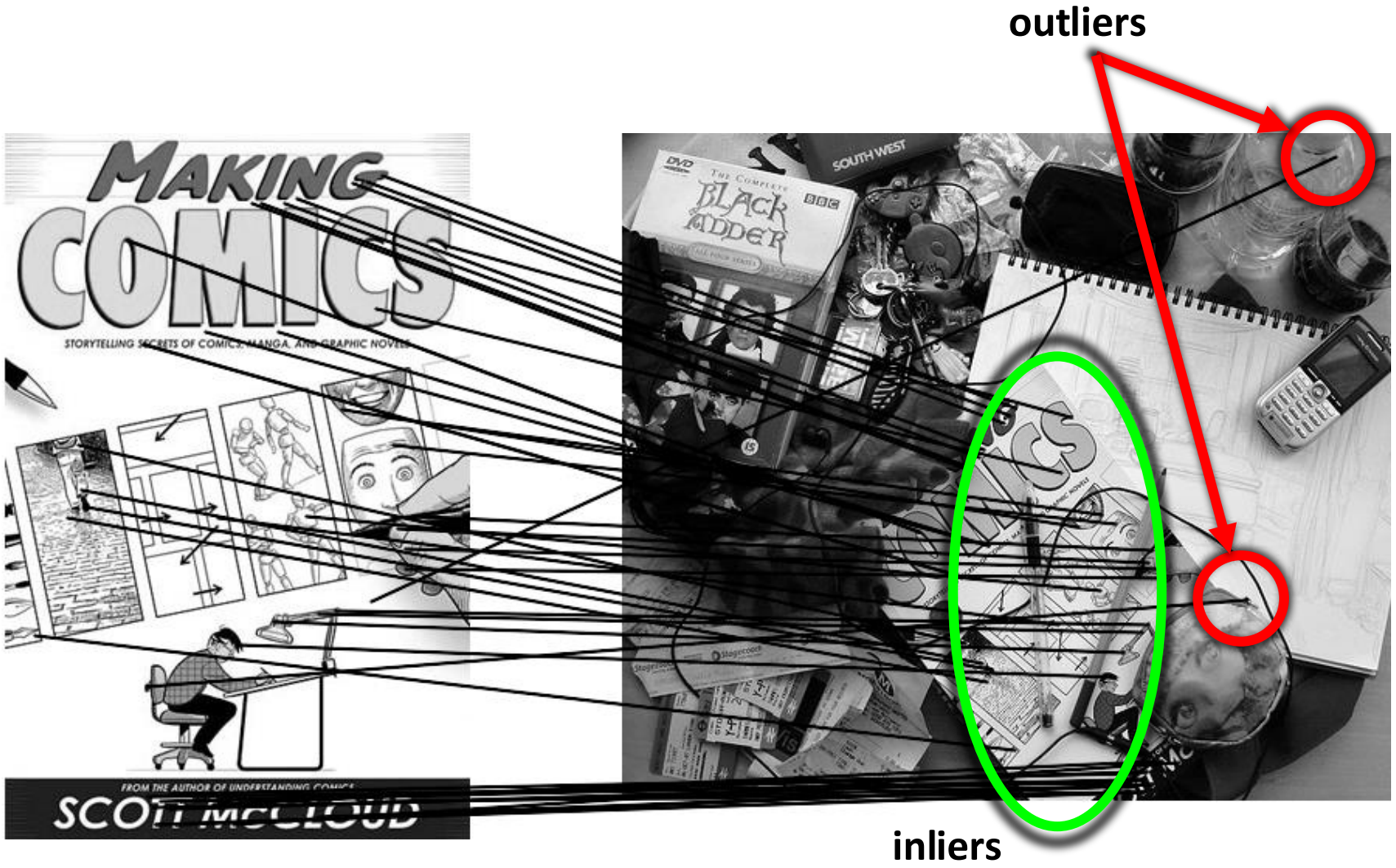
# How does least squares do?

Error based on **squared** residual

Very bad at handling outliers in data



# Outliers


outliers
MAKING
COMICS
STORYTELLING SECRETS OF COMICS, MANGA, AND GRAPHIC NOVELS
FROM THE AUTHOR OF UNDERSTANDING COMICS
SCOTT McCLOUD
DVD
THE COMPLETE
BLACK ADDER
BBC
SOUTH WEST
inliers

# Homography computation

- The homography computation must be robust enough to deal with
  - errors in the detection of some corners
  - wrong matches
  - occluded corners

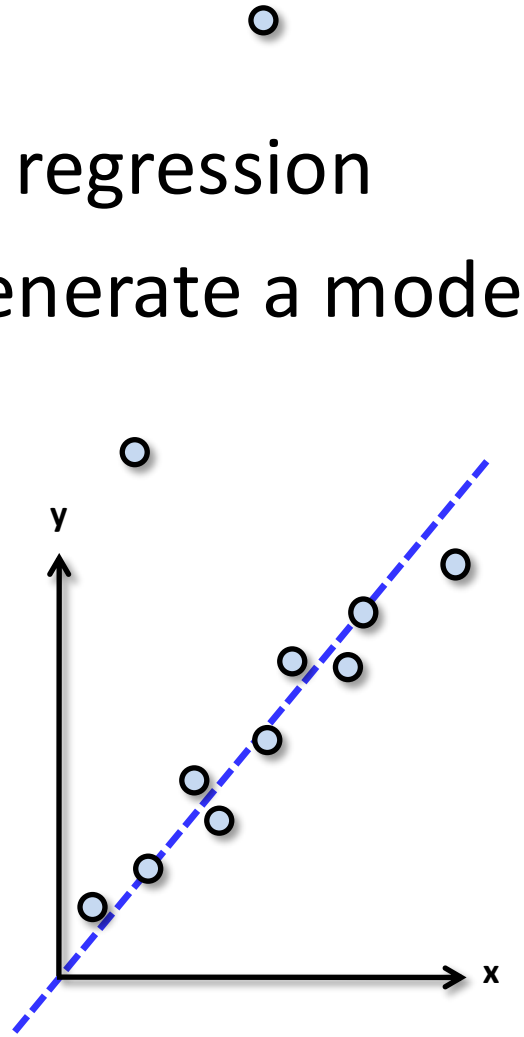
- For that, *RANSAC* may be used

# RANSAC - RAndom SAmple Consensus

- Idea:
  - All the inliers will agree with each other
  - The (hopefully small) number of outliers will likely disagree with each other, since they are mostly random
  - We just need to test different model hypothesis

- RANSAC only has guarantees if there are < 50% outliers

# RANSAC



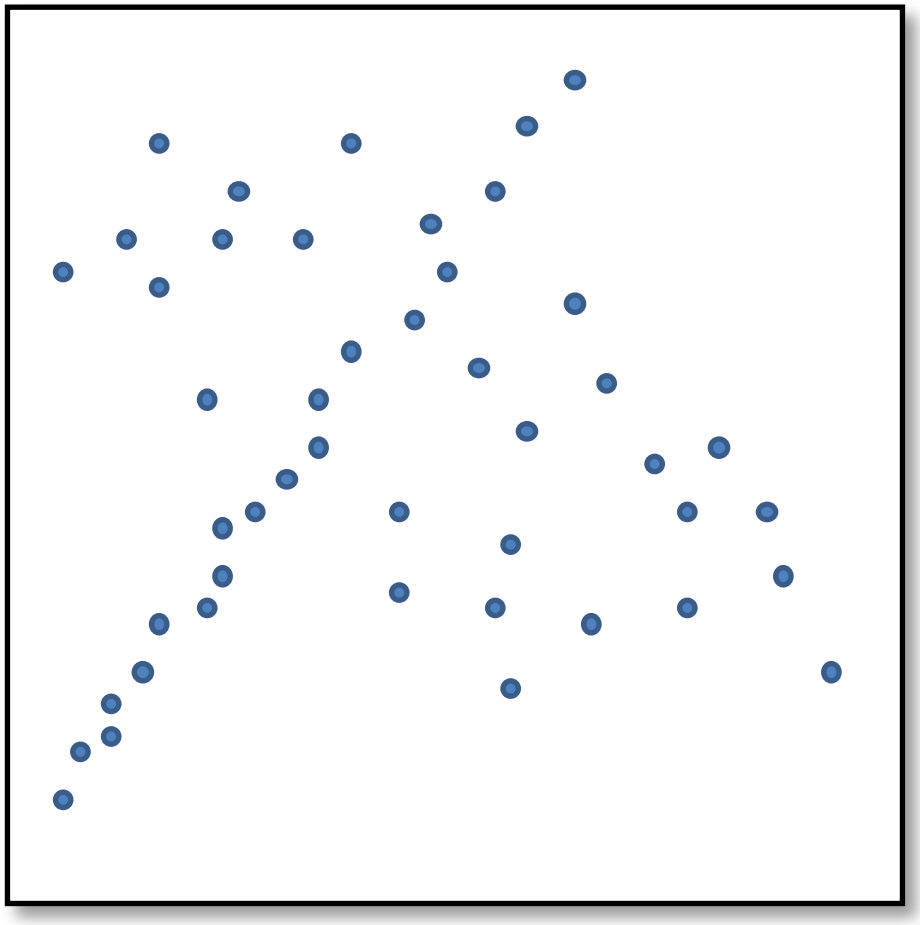

- Back to linear regression
- How do we generate a model hypothesis?

# RANSAC

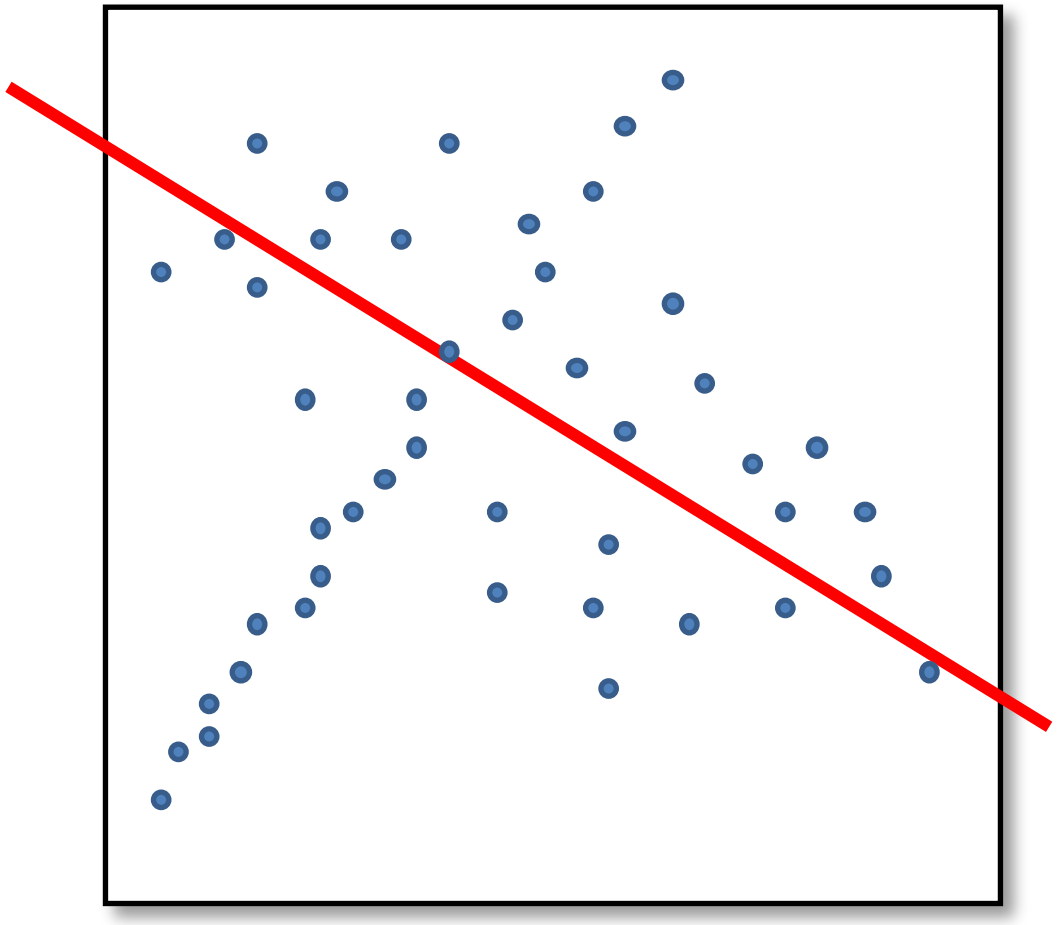
- Given a hypothesized line
- Count the number of points that “agree” with the line
  - “Agree” = within a small distance of the line
  - I.e., the **inliers** to that line

- For all possible lines, select the one with the largest number of inliers

# Counting inliers

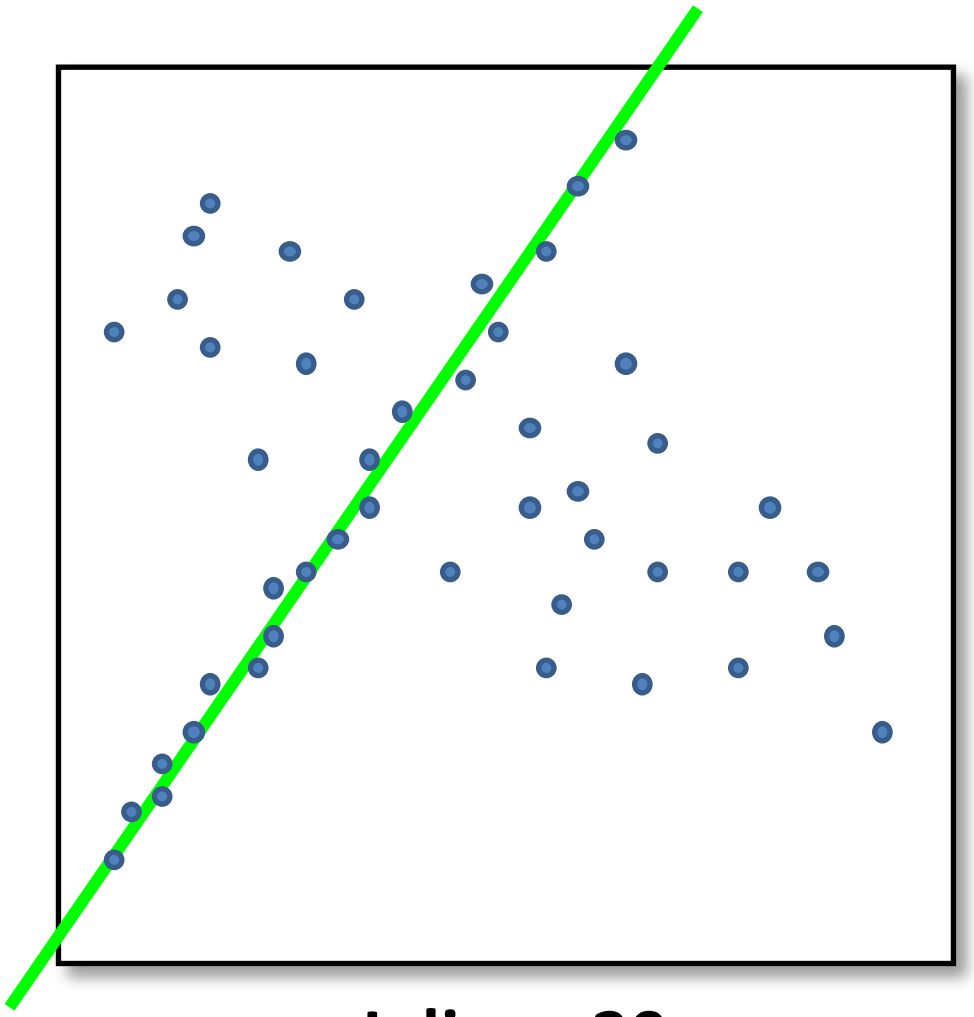


# Counting inliers



**Inliers: 3**

# Counting inliers



**Inliers: 20**

# RANSAC

- General version:
  1. Randomly choose *s* samples
     - Typically *s* = minimum sample size that lets you fit a model
  2. Fit a model (e.g., line) to those samples
  3. Count the number of inliers that approximately fit the model
  4. Repeat *N* times
  5. Choose the model that has the largest set of inliers

# RANSAC parameters

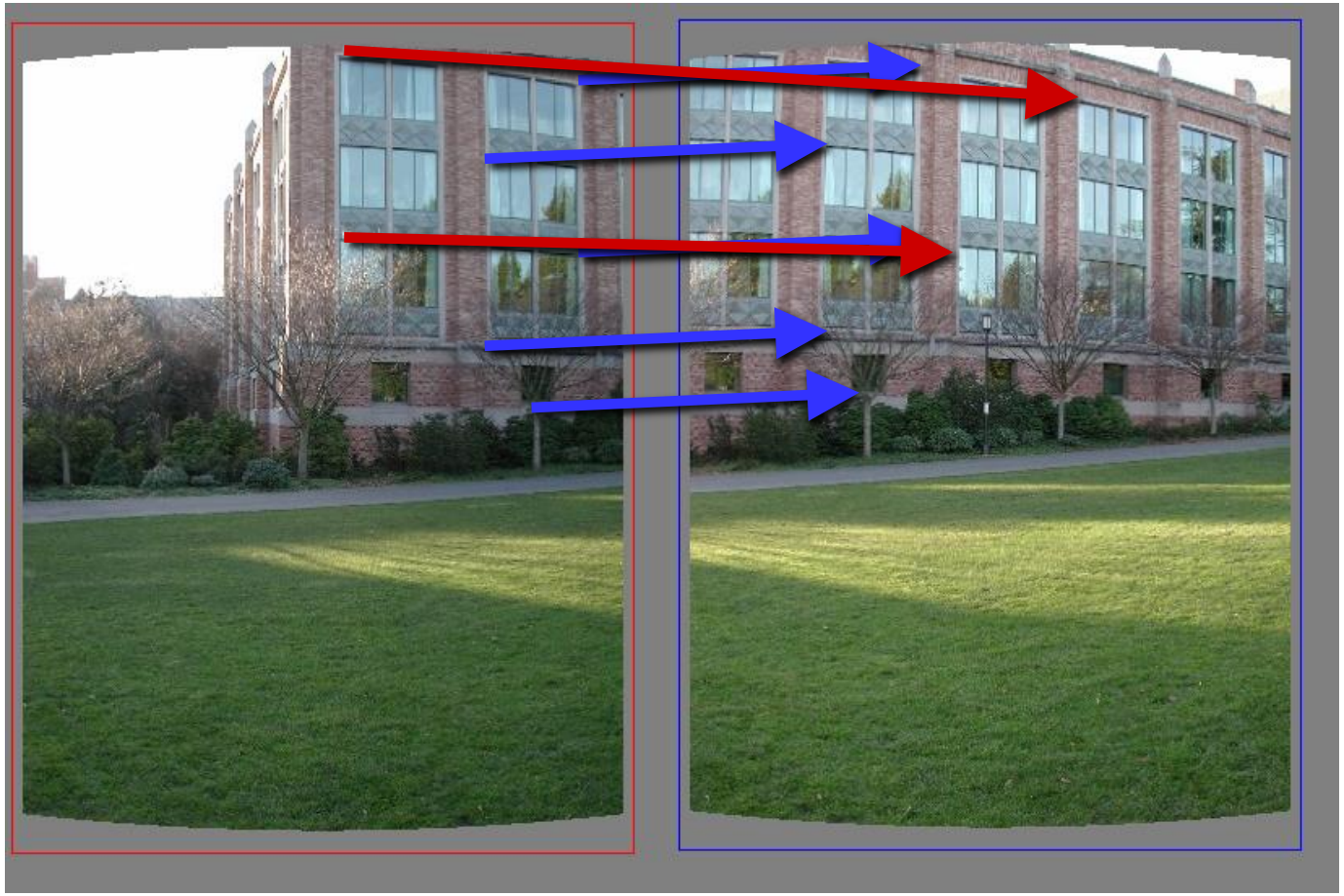
- **Inlier threshold** related to the amount of noise we expect in inliers
  - Often model noise as Gaussian with some standard deviation (e.g., 3 pixels)

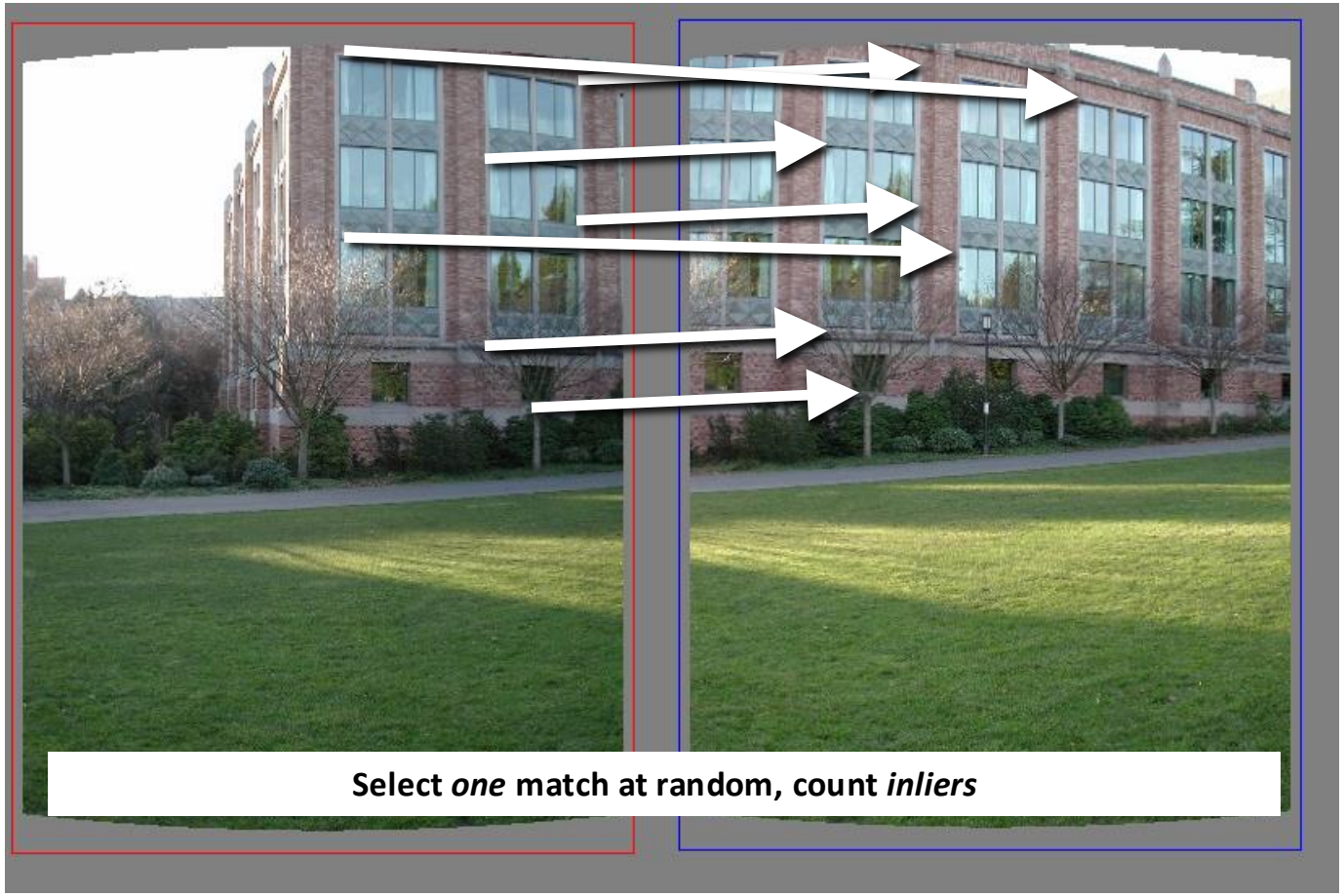
- **Number of iterations**
  - related to the percentage of outliers we expect, and the probability of success we would like to guarantee

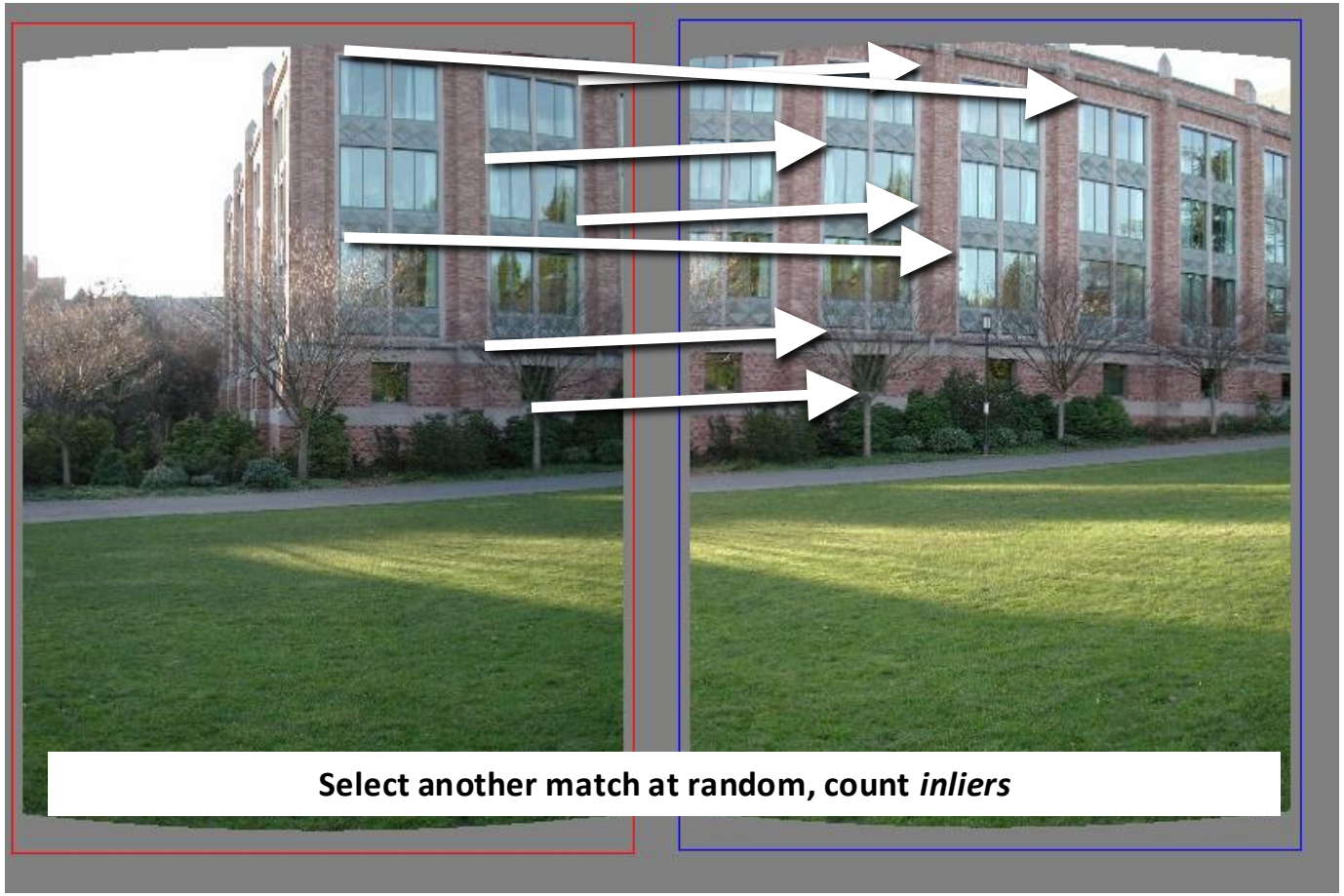
- Other parameters: model, n points to fit model

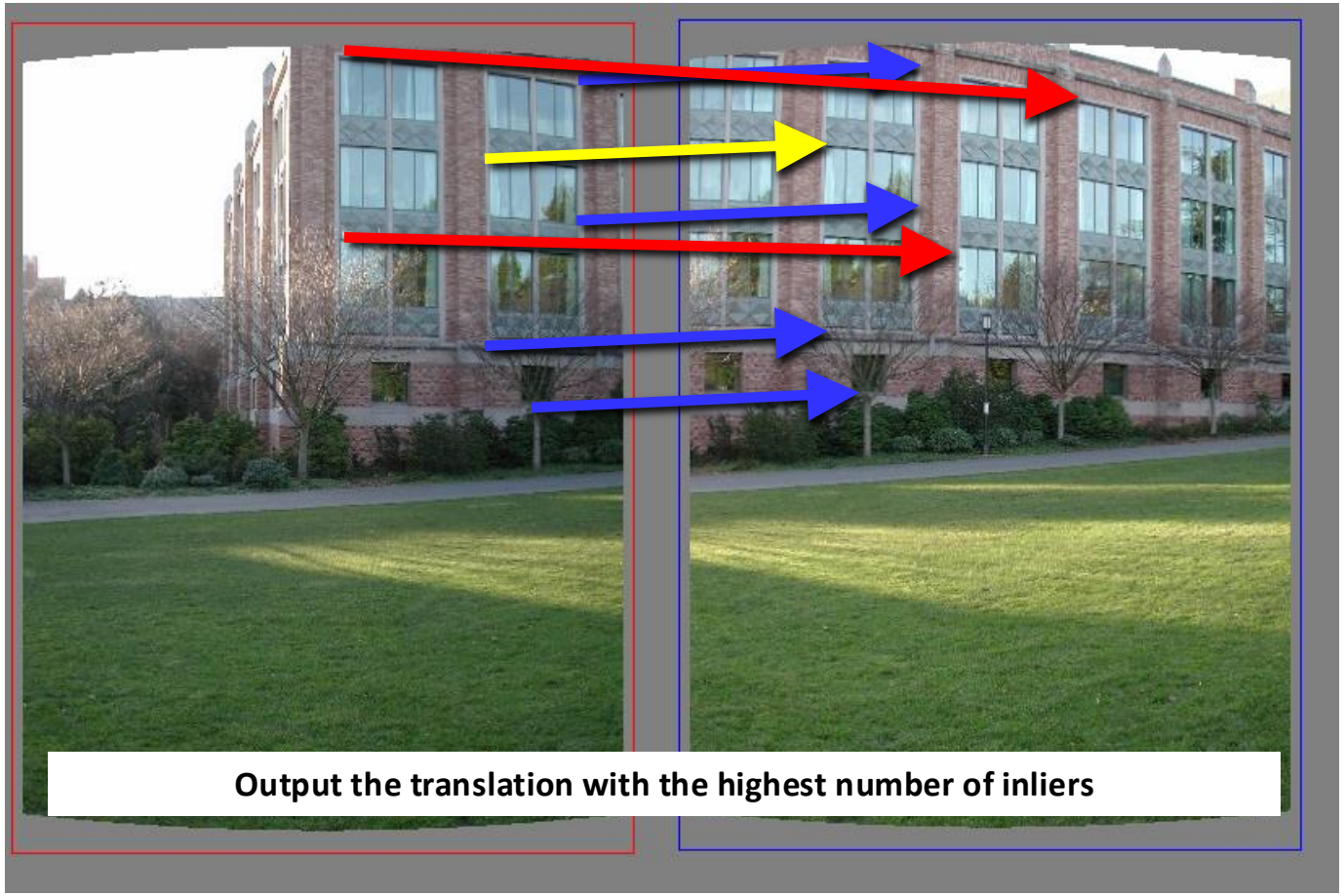
# RANSAC to estimate transformation

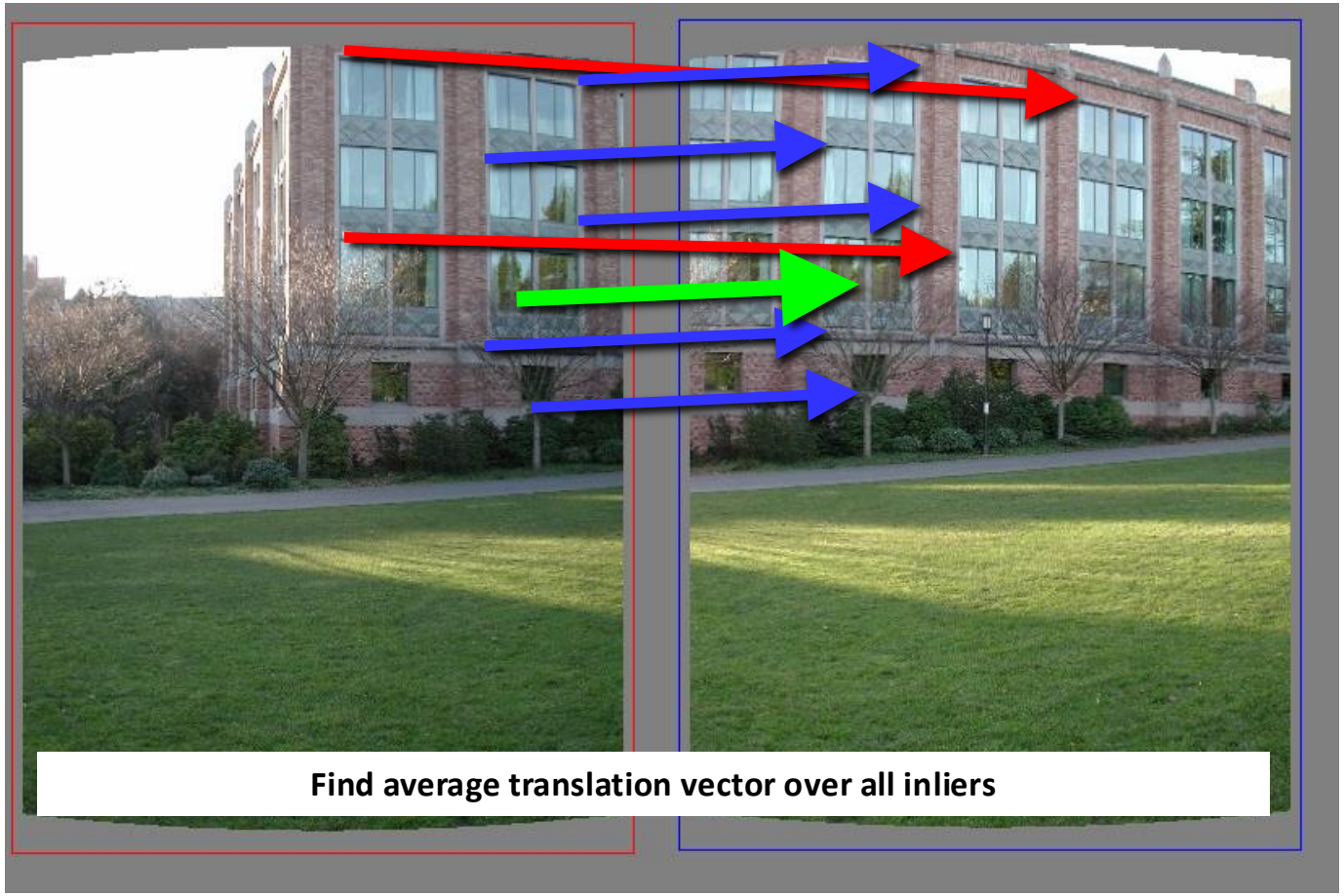
Let’s consider only translations




Select *one* match at random, count *inliers*


Select another match at random, count *inliers*


Output the translation with the highest number of inliers


Find average translation vector over all inliers

# Homography computation

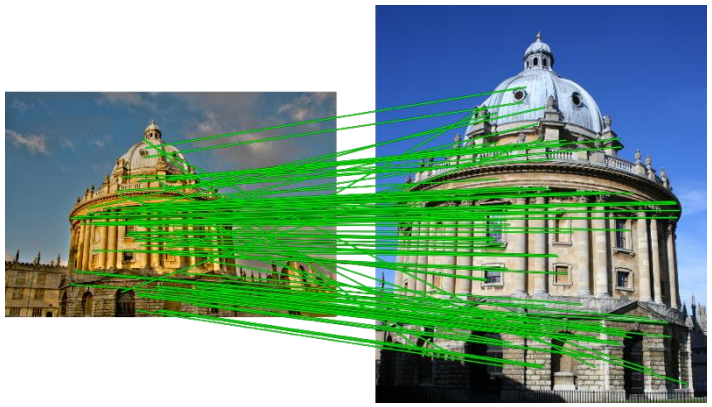
**The *Model* is the *Homography***

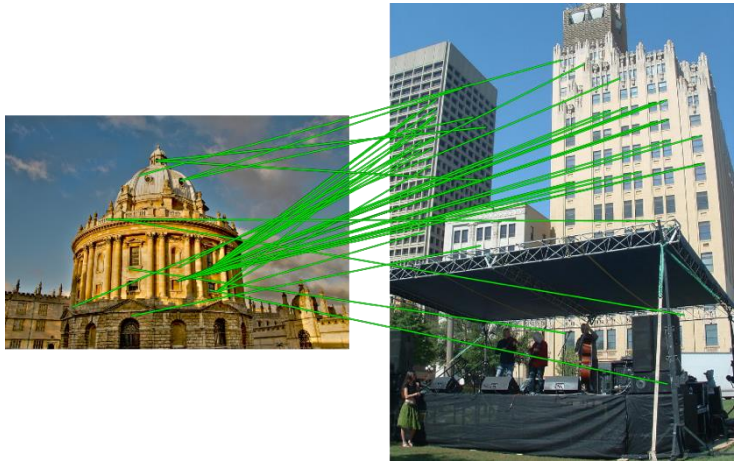
1. Randomly select 4 matched keypoints from 2 images
2. Calculate the homography H with those 4 points $\{[x, y, 1]^T \leftrightarrow [x', y', 1]^T\}$
3. For all matched points, calculate the distance between $[x, y, 1]^T$ and $[H]\ [x', y', 1]^T$ , D
4. Count the number of matched pair of points for which he distance in below a threshold
   a. If D < threshold → considered to be *inliers* ($I_H$ = number of *inliers*)
   b. If D > threshold → considered to be *outliers*
5. Repeat steps 1 to 4 N times

1. Chose the model with largest number of *inliers*
2. (optional final) Recalculate the homography using only the *inliers.*

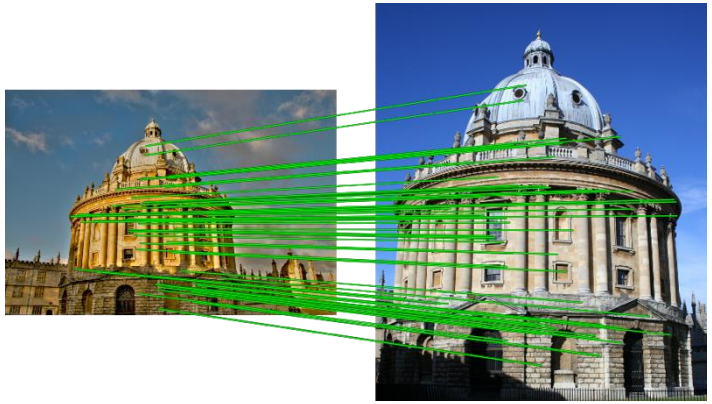
# Using RANSAC to verify matches

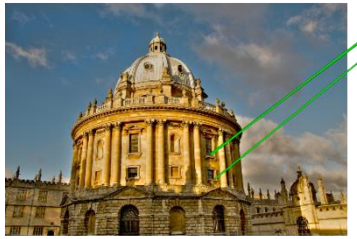
No verification





With RANSAC verification



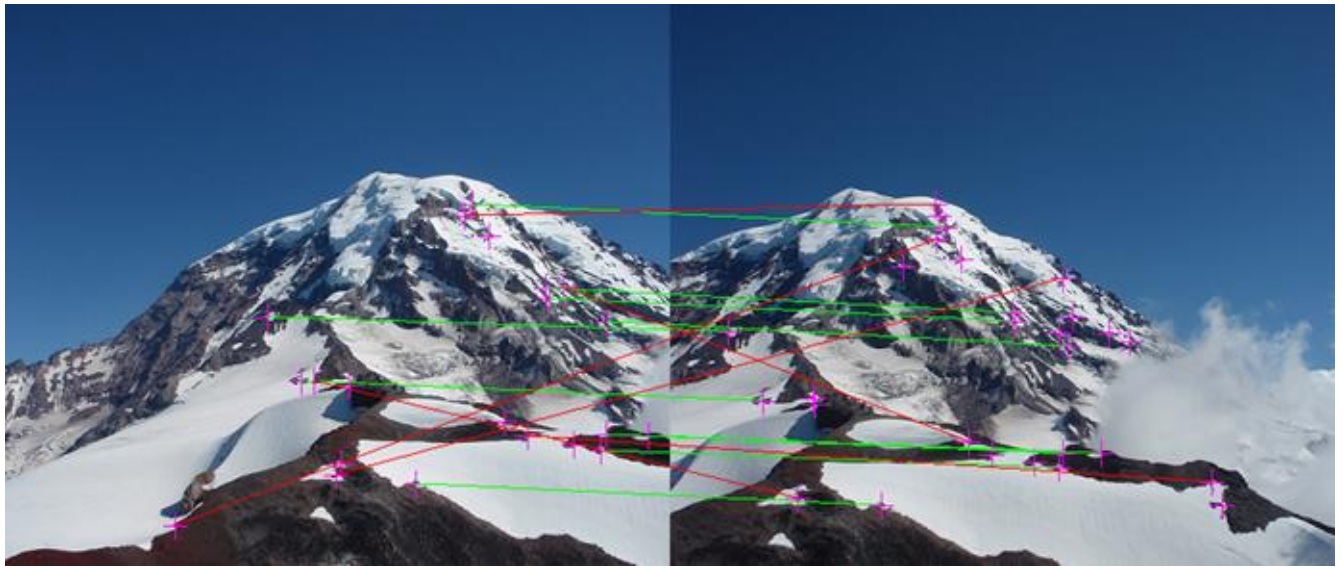


Fails to meet threshold on #inliers

# Use case: panorama

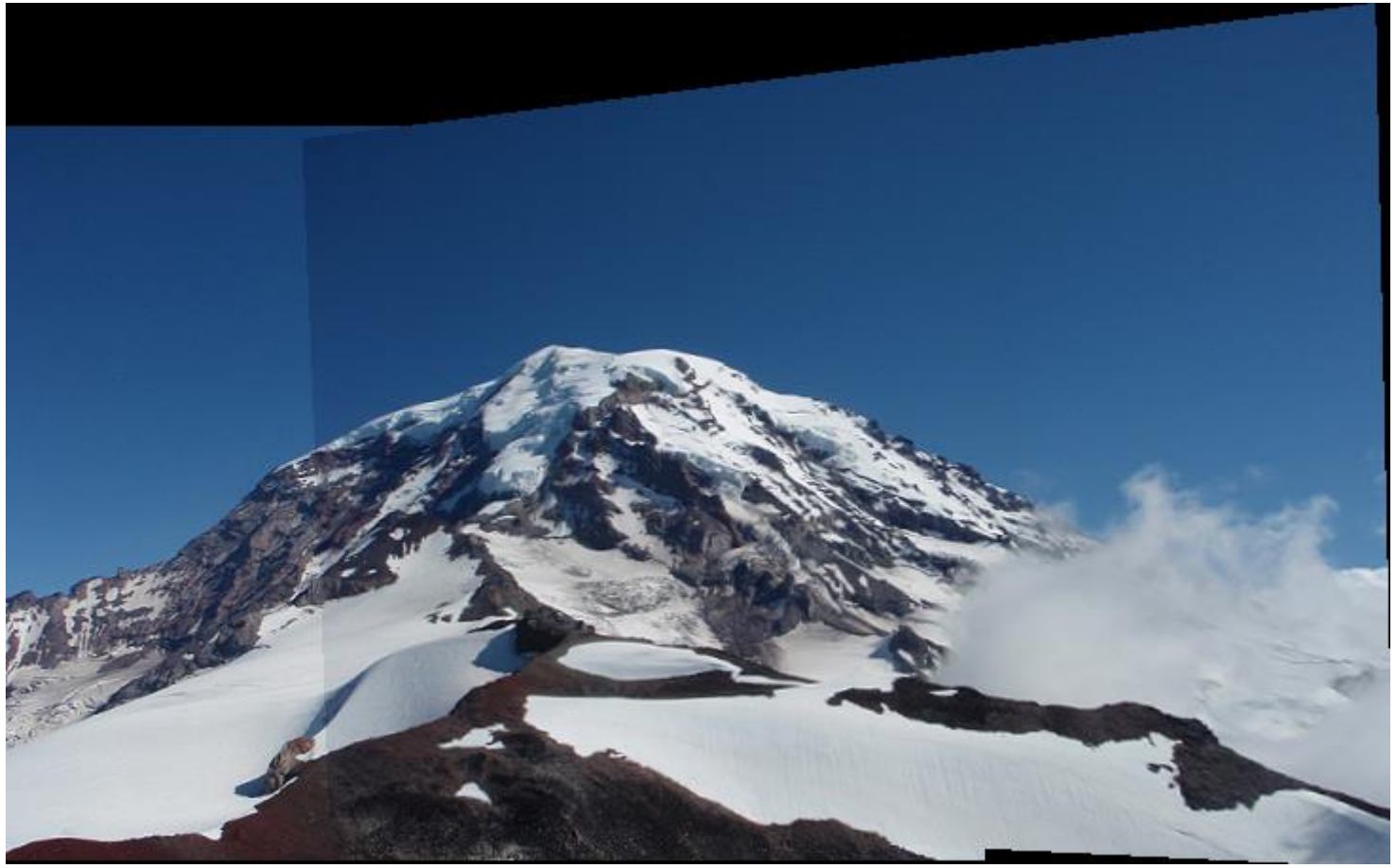
What pipeline can we use?

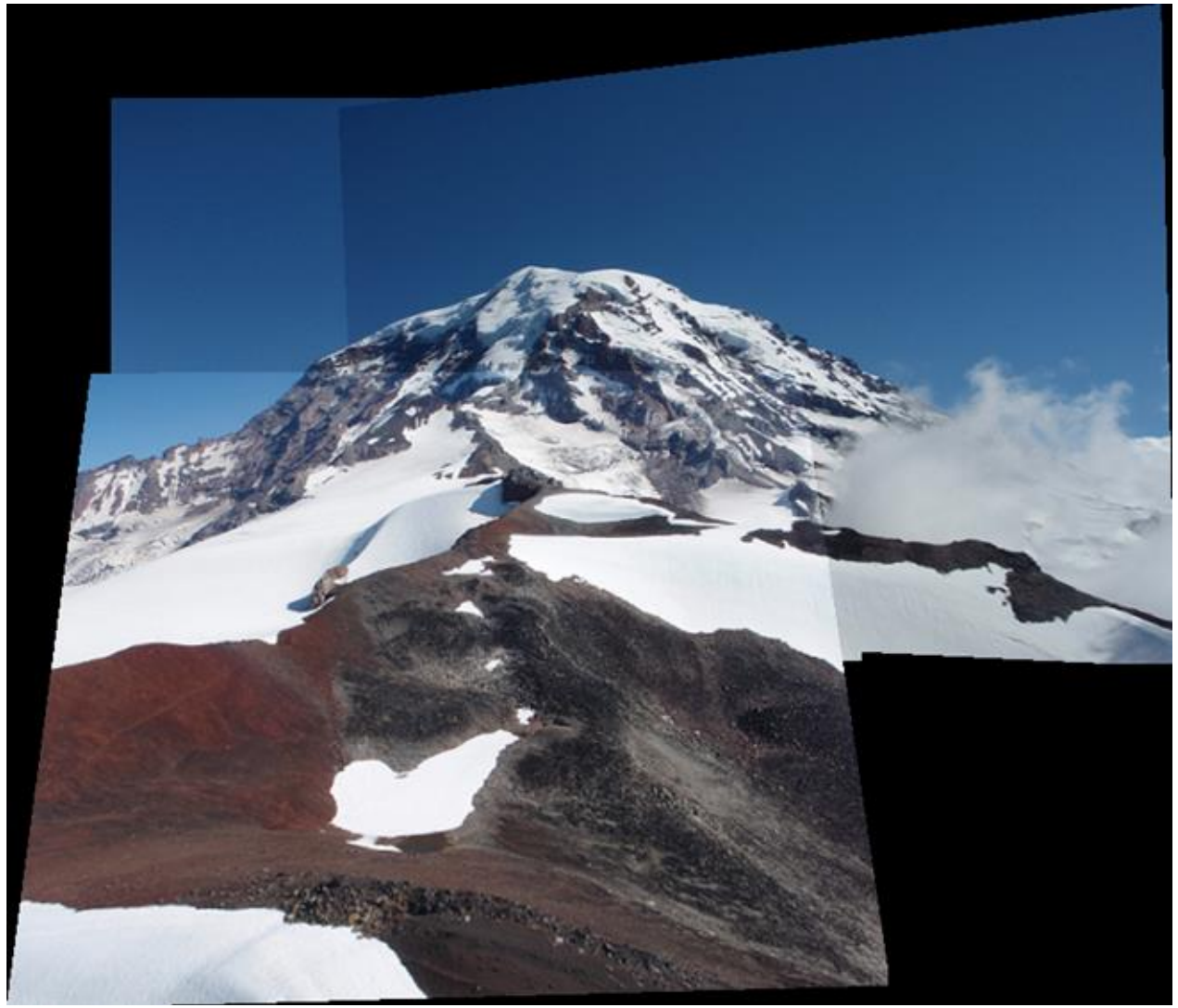
- Find **local features** in both images
- Calculate **descriptors**
- **Match** descriptors
- Calculate **homography** (e.g. RANSAC)
- **Stitch together** images with homography

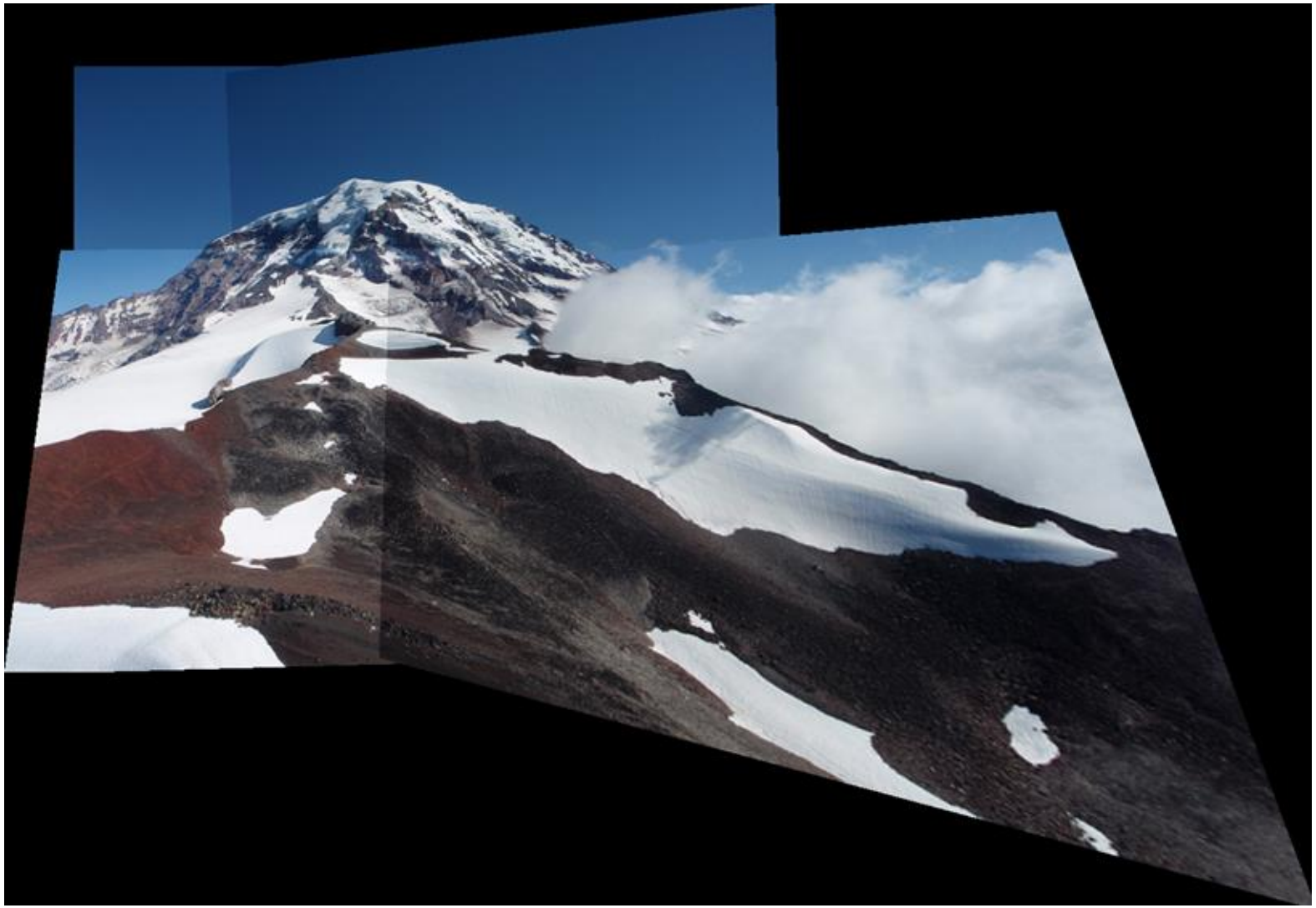


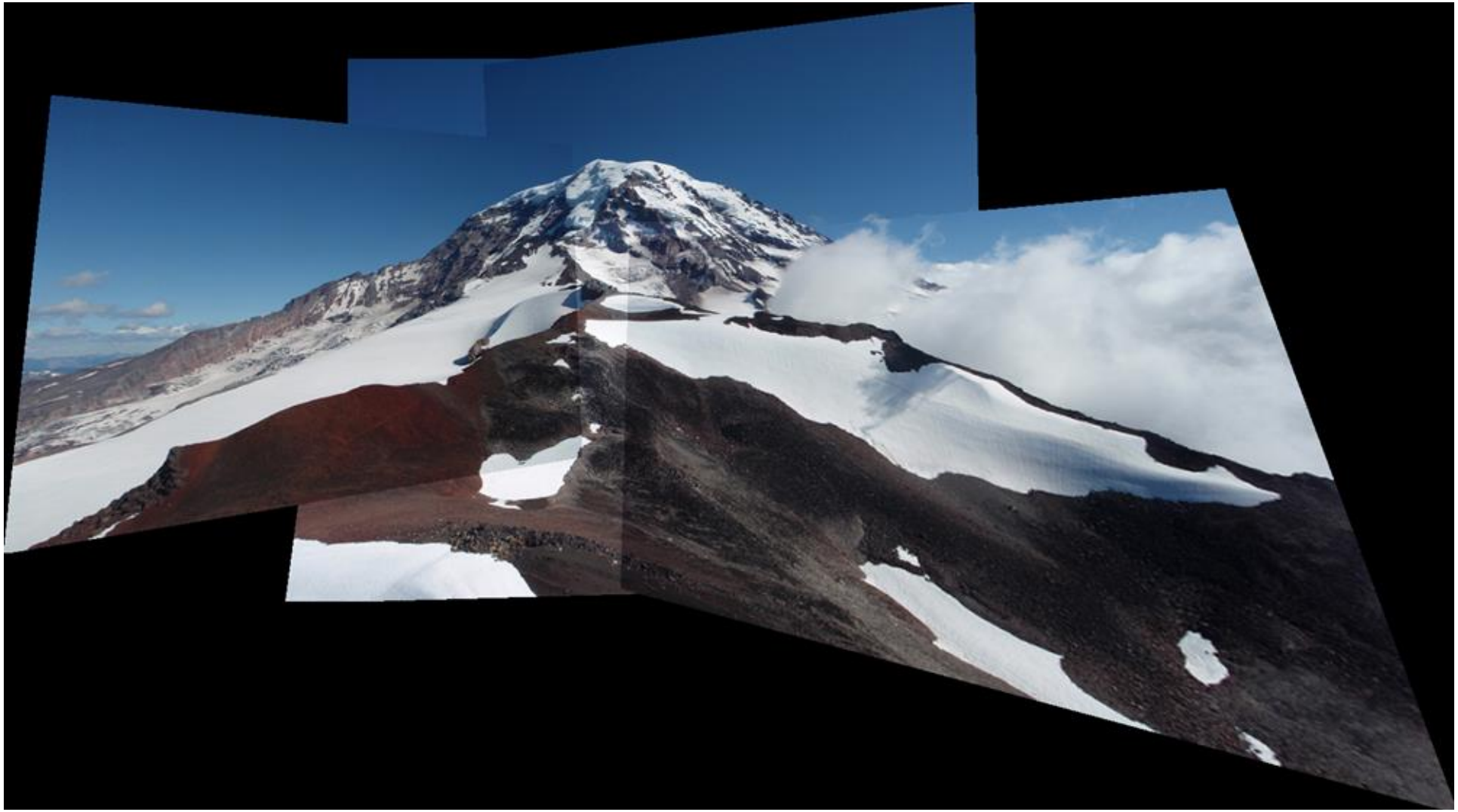
- More Images?

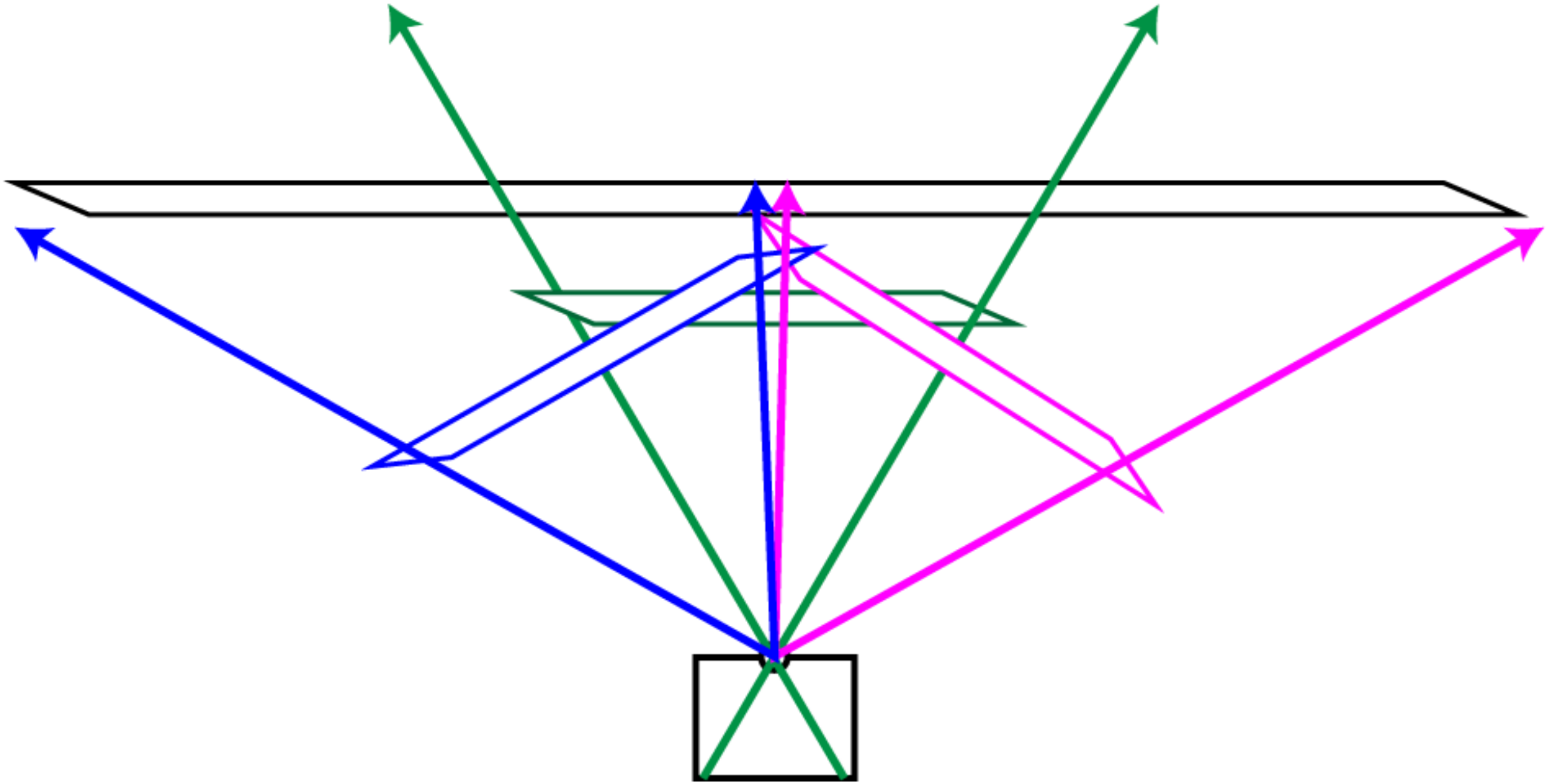
via JoeRedmon



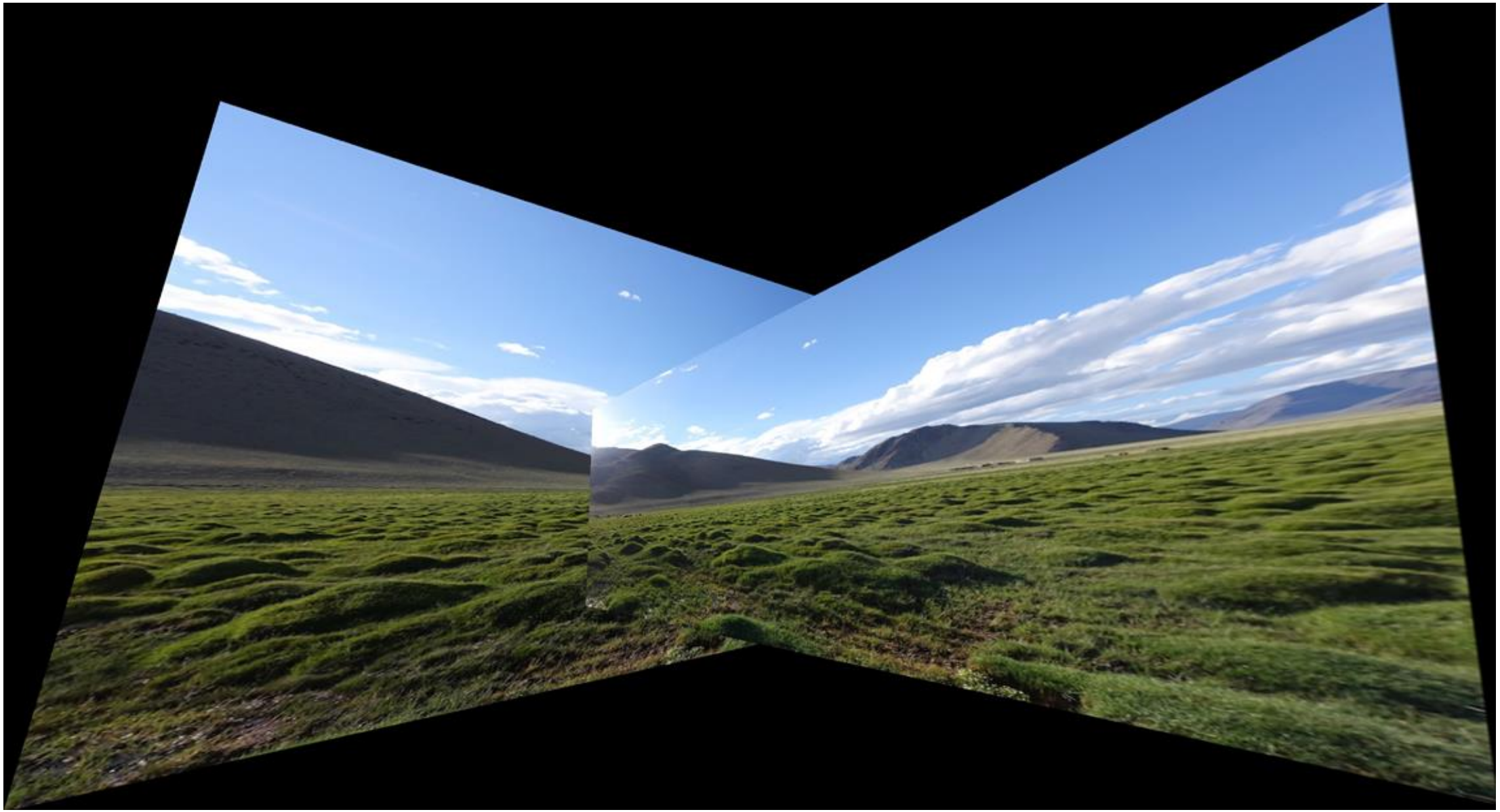








# May not work well for big panoramas



# Possible solution: cylindrical projection